



UNINASSAU PAULISTA

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Discente:

Rafael Moises, Mayk Dias, Elizeu Da Silva, José Rennan e Anderson Trindade

Docente:

Victor Henrique Dos Santos

SISTEMA DE GESTÃO DE EVENTOS ACADÊMICOS

Paulista – PE

2026

SUMÁRIO

- 1 - INTRODUÇÃO
- 2 - CONTEXTUALIZAÇÃO DO PROBLEMA
- 3 - TEMA E OBJETIVOS
- 4 - ESCOPO DO SISTEMA
- 5 - REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS
- 6 - PERFIS DE USUÁRIO
- 7 - REGRAS DE NEGÓCIO
- 8 - ENTIDADES E DICIONÁRIO DE DADOS
- 9 - RELACIONAMENTOS E MODELO DO BANCO
- 10 - API: ROTAS, CONTRATOS E STATUS HTTP
- 11 - ARQUITETURA E ESTRUTURA DO PROJETO
- 12 - TECNOLOGIAS E JUSTIFICATIVAS
- 13 - MATRIZ DE PERMISSÕES
- 14 - BANCO DE DADOS E INTEGRIDADE
- 15 - FUNCIONALIDADES IMPLEMENTADAS
- 16 - TESTES E VALIDAÇÃO
- 17 - BACKLOG
- 18 - CRITÉRIOS DE ACEITAÇÃO
- 19 - SITUAÇÃO ATUAL E PRÓXIMAS EVOLUÇÕES
- 20 - CONCLUSÃO
- 21 - REFERÊNCIAS

1. INTRODUÇÃO

O Sistema de Gestão de Eventos Acadêmicos é uma API REST desenvolvida para organizar informações relacionadas à realização de eventos no ambiente acadêmico. A solução centraliza o gerenciamento de eventos, categorias, usuários, organizadores, inscrições e certificados.

O projeto foi desenvolvido com FastAPI e utiliza PostgreSQL como banco de dados relacional, com SQLAlchemy como camada de integração entre a aplicação Python e o banco. A documentação e os testes dos endpoints são realizados por meio do Swagger/OpenAPI disponibilizado pela FastAPI.

Esta documentação consolida a situação atual do projeto, incluindo requisitos, regras de negócio, entidades, relacionamentos, rotas, contratos, arquitetura, banco de dados, testes e critérios de aceitação.

2. CONTEXTUALIZAÇÃO DO PROBLEMA

Em atividades acadêmicas, eventos como palestras, workshops, minicursos e outras atividades precisam de informações organizadas sobre datas, horários, locais, categorias, responsáveis, participantes e certificados. Quando esses dados são controlados de forma descentralizada, podem ocorrer duplicidades, inscrições acima da capacidade, inconsistências entre entidades e dificuldades para acompanhar certificados.

A API proposta busca solucionar esse problema por meio de uma estrutura centralizada e persistente, permitindo que as operações sejam realizadas por endpoints REST e que regras de negócio sejam aplicadas antes da gravação ou alteração dos dados.

3. TEMA E OBJETIVOS

3.1 Tema

Sistema de Gestão de Eventos Acadêmicos.

3.2 Objetivo geral

Desenvolver uma API REST para gerenciamento de eventos acadêmicos, permitindo controlar eventos, categorias, usuários, organizadores, inscrições e certificados com persistência em banco de dados relacional e aplicação de regras de negócio.

3.3 Objetivos específicos

- Permitir o cadastro, consulta, atualização e exclusão de eventos.
- Organizar os eventos por categorias.
- Gerenciar usuários e organizadores.
- Permitir inscrições de usuários em eventos.
- Controlar o limite de vagas dos eventos.
- Emitir certificados vinculados a inscrições confirmadas.
- Evitar registros duplicados e referências inexistentes.
- Disponibilizar documentação e testes dos endpoints pelo Swagger/OpenAPI.
- Manter os dados persistidos no PostgreSQL.

4. ESCOPO DO SISTEMA

- Gerenciamento de eventos acadêmicos.
- Gerenciamento de categorias.
- Gerenciamento de usuários.
- Gerenciamento de inscrições.
- Gerenciamento de certificados.
- Gerenciamento de organizadores.

- Validações de entrada e regras de negócio.
- Persistência e relacionamentos no PostgreSQL.
- Documentação dos endpoints com Swagger/OpenAPI.

A autenticação e autorização por JWT podem ser incorporadas como evolução posterior. Os perfis e a matriz de permissões estão definidos nesta documentação para orientar essa evolução.

5. REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS

5.1 Requisitos funcionais

Código	Requisito	Descrição
RF01	Cadastrar eventos acadêmicos.	O sistema deve permitir informar título, descrição, data, horário, local, categoria, organizador e quantidade de vagas.
RF02	Consultar eventos.	O sistema deve permitir listar e consultar eventos por ID.
RF03	Atualizar eventos.	O sistema deve permitir alterar dados de um evento respeitando as regras de negócio.
RF04	Excluir eventos.	O sistema deve impedir a exclusão de evento que possua inscrições.
RF05	Gerenciar categorias.	O sistema deve permitir listar, cadastrar, atualizar e excluir categorias respeitando

		vínculos.
RF06	Gerenciar usuários.	O sistema deve permitir listar, cadastrar, atualizar e excluir usuários respeitando vínculos.
RF07	Gerenciar inscrições.	O sistema deve permitir listar, cadastrar, atualizar e excluir inscrições conforme as regras.
RF08	Controlar vagas.	O sistema deve impedir inscrições acima da capacidade do evento.
RF09	Gerenciar certificados.	O sistema deve permitir listar, cadastrar, atualizar e excluir certificados conforme as regras.
RF10	Gerenciar organizadores.	O sistema deve permitir listar, cadastrar, atualizar e excluir organizadores respeitando vínculos.
RF11	Validar referências.	O sistema deve impedir o uso de IDs de usuário, evento, categoria, organizador ou inscrição inexistentes.
RF12	Documentar a API.	Os endpoints devem ser disponibilizados pelo Swagger/OpenAPI.

5.2 Requisitos não funcionais

Código	Requisito	Descrição
RNF01	Tecnologia	A API deve ser desenvolvida em Python utilizando FastAPI.
RNF02	Persistência	Os dados devem ser armazenados no PostgreSQL.
RNF03	ORM	A integração Python/PostgreSQL deve utilizar SQLAlchemy.
RNF04	Validação	Os dados de entrada devem ser validados pelos modelos Pydantic e pelas regras de negócio.
RNF05	Documentação	A API deve possuir documentação interativa via Swagger/OpenAPI.
RNF06	Manutenibilidade	O projeto deve manter separação entre rotas, modelos Pydantic, configuração do banco e modelos ORM.
RNF07	Integridade	As relações entre entidades devem ser protegidas por chaves estrangeiras e restrições de integridade.
RNF08	Tratamento de erros	A API deve retornar códigos

		HTTP coerentes e mensagens de erro compreensíveis.
--	--	--

6. PERFIS DE USUÁRIO

Os perfis abaixo seguem a estrutura proposta para o projeto. A implementação de autenticação/autorização poderá ser adicionada posteriormente.

Perfil	Responsabilidade
Administrador	Gerenciar dados gerais do sistema, categorias, usuários, organizadores e eventos.
Organizador	Gerenciar eventos sob sua responsabilidade e acompanhar inscrições.
Participante	Consultar eventos, realizar inscrições e acompanhar sua participação e certificados.

7. REGRAS DE NEGÓCIO

1. RN01 – O e-mail de usuário deve ser único.
2. RN02 – O e-mail de organizador deve ser único.
3. RN03 – O nome da categoria deve ser único.
4. RN04 – Um evento deve possuir pelo menos uma vaga.
5. RN05 – A data de um evento não pode ser anterior à data atual.
6. RN06 – A categoria informada no evento deve existir.
7. RN07 – O organizador informado no evento deve existir.
8. RN08 – Um usuário não pode possuir duas inscrições para o mesmo evento.
9. RN09 – O número de inscrições ativas não pode ultrapassar a quantidade de vagas.
10. RN10 – Inscrição deve possuir usuário e evento existentes.
11. RN11 – O status de inscrição deve ser Pendente, Confirmada ou Cancelada.

12. RN12 – Inscrições canceladas não consomem a capacidade ativa do evento.
13. RN13 – Certificado somente pode ser emitido para inscrição Confirmada.
14. RN14 – Uma inscrição não pode possuir mais de um certificado.
15. RN15 – O código do certificado deve ser único.
16. RN16 – Um evento que possua inscrições não pode ser excluído.
17. RN17 – Uma categoria vinculada a evento não pode ser excluída.
18. RN18 – Um usuário que possua inscrições não pode ser excluído.
19. RN19 – Um organizador vinculado a eventos não pode ser excluído.
20. RN20 – Uma inscrição que possua certificado não pode ser excluída.
21. RN21 – A quantidade de vagas de um evento não pode ser reduzida para valor menor que o número de inscrições ativas.
22. RN22 – Não deve ser possível criar uma inscrição para usuário ou evento inexistente.

8. ENTIDADES E DICIONÁRIO DE DADOS

O modelo atual possui seis entidades principais: Evento, Categoria, Usuário, Inscrição, Certificado e Organizador.

8.1 Evento

Campo	Tipo	Regra/Função
id	Integer	PK; identificador do evento.
titulo	Varchar(150)	Título do evento; obrigatório.
descricao	Text	Descrição; obrigatória.
data	Varchar(10)	Data no formato AAAA-MM-DD; obrigatória.
horario	Varchar(5)	Horário; obrigatório.
local	Varchar(150)	Local do evento; obrigatório.
categoria_id	Integer	FK para categorias.id;

		obrigatório.
organizador_id	Integer	FK para organizadores.id; obrigatório.
vagas	Integer	Quantidade de vagas; deve ser maior que zero.

8.2 Categoria

Campo	Tipo	Regra/Função
id	Integer	PK.
nome	Varchar(100)	Nome da categoria; único e obrigatório.

8.3 Usuário

Campo	Tipo	Regra/Função
id	Integer	PK.
nome	Varchar(150)	Nome do usuário; obrigatório.
email	Varchar(150)	E-mail; único e obrigatório.
tipo	Varchar(50)	Tipo/perfil do usuário.

8.4 Inscrição

Campo	Tipo	Regra/Função
id	Integer	PK.
usuario_id	Integer	FK para usuarios.id; obrigatório.
evento_id	Integer	FK para eventos.id; obrigatório.
status	Varchar(50)	Pendente, Confirmada ou Cancelada.

8.5 Certificado

Campo	Tipo	Regra/Função
id	Integer	PK.
inscricao_id	Integer	FK para inscricoes.id; obrigatório.
codigo	Varchar(100)	Código único do certificado.
data_emissao	Varchar(10)	Data de emissão.

8.6 Organizador

Campo	Tipo	Regra/Função
id	Integer	PK.
nome	Varchar(150)	Nome do organizador; obrigatório.
email	Varchar(150)	E-mail; único e obrigatório.
tipo	Varchar(50)	Tipo do organizador.

9. RELACIONAMENTOS E MODELO DO BANCO

Origem	Relacionamento	Destino	Cardinalidade
categorias.id	1:N	eventos.categoria_id	Uma categoria pode estar em vários eventos.
organizadores.id	1:N	eventos.organizador_id	Um organizador pode estar associado a vários eventos.
usuarios.id	1:N	inscricoes.usuario_id	Um usuário pode possuir várias inscrições.

eventos.id	1:N	inscricoes.evento_id	Um evento pode possuir várias inscrições.
inscricoes.id	1:0..1	certificados.inscricao_id	Uma inscrição pode ter no máximo um certificado.

Chaves estrangeiras implementadas no banco:

- eventos.categoria_id → categorias.id
- eventos.organizador_id → organizadores.id
- inscricoes.usuario_id → usuarios.id
- inscricoes.evento_id → eventos.id
- certificados.inscricao_id → inscricoes.id

Representação textual do DER:

CATEGORIAS (1) — (N) EVENTOS (1) — (N) INSCRIÇÕES (1) —
 (0..1) CERTIFICADOS
 ORGANIZADORES (1) — (N) EVENTOS
 USUÁRIOS (1) — (N) INSCRIÇÕES

10. API: ROTAS, CONTRATOS E STATUS HTTP

10.1 Endpoint raiz

Método	Rota	Resposta
GET	/	Mensagem de funcionamento da API.

10.2 Eventos

Método	Rota	Função	Sucesso
GET	/eventos	Listar eventos	200
POST	/eventos	Cadastrar evento	201
GET	/eventos/{evento_id}	Consultar evento	200
PUT	/eventos/{evento_id}	Atualizar evento	200
DELETE	/eventos/{evento_id}	Excluir evento	204

10.3 Categorias

Método	Rota	Função	Sucesso
GET	/categorias	Listar categorias	200
POST	/categorias	Cadastrar categoria	201
GET	/categorias/{categoria_id}	Consultar categoria	200
PUT	/categorias/{categoria_id}	Atualizar categoria	200
DELETE	/categorias/{categoria_id}	Excluir categoria	204

10.4 Usuários

Método	Rota	Função	Sucesso
GET	/usuarios	Listar usuários	200
POST	/usuarios	Cadastrar usuário	201
GET	/usuarios/{usuario_id}	Consultar usuário	200
PUT	/usuarios/{usuario_id}	Atualizar usuário	200
DELETE	/usuarios/{usuario_id}	Excluir usuário	204

	{usuario_id}		
--	--------------	--	--

10.5 Inscrições

Método	Rota	Função	Sucesso
GET	/inscricoes	Listar inscrições	200
POST	/inscricoes	Cadastrar inscrição	201
GET	/inscricoes/ {inscricao_id}	Consultar inscrição	200
PUT	/inscricoes/ {inscricao_id}	Atualizar inscrição	200
DELETE	/inscricoes/ {inscricao_id}	Excluir inscrição	204

10.6 Certificados

Método	Rota	Função	Sucesso
GET	/certificados	Listar certificados	200
POST	/certificados	Emitir certificado	201
GET	/certificados/ {certificado_id}	Consultar certificado	200
PUT	/certificados/ {certificado_id}	Atualizar certificado	200
DELETE	/certificados/ {certificado_id}	Excluir certificado	204

10.7 Organizadores

Método	Rota	Função	Sucesso
GET	/organizadores	Listar organizadores	200
POST	/organizadores	Cadastrar organizador	201

GET	/organizadores/ {organizador_id}	Consultar organizador	200
PUT	/organizadores/ {organizador_id}	Atualizar organizador	200
DELETE	/organizadores/ {organizador_id}	Excluir organizador	204

10.8 Contratos de entrada – exemplos

POST /eventos:

```
{ "titulo": "Palestra sobre Inteligência Artificial", "descricao": "Evento acadêmico sobre IA", "data": "2026-10-10", "horario": "19:00", "local": "Auditório Principal", "categoria_id": 5, "organizador_id": 2, "vagas": 100 }
```

POST /categorias:

```
{ "nome": "Workshop" }
```

POST /usuarios:

```
{ "nome": "Rafael", "email": "rafael@example.com", "tipo": "Participante" }
```

POST /inscricoes:

```
{ "usuario_id": 1, "evento_id": 1, "status": "Confirmada" }
```

POST /certificados:

```
{ "inscricao_id": 3, "codigo": "CERT-003", "data_emissao": "2026-10-03" }
```

POST /organizadores:

```
{ "nome": "Organizador do Evento", "email": "organizador@example.com", "tipo": "Organizador" }
```

10.9 Códigos HTTP e tratamento de erros

Código	Uso no projeto
--------	----------------

200 OK	Consulta e atualização realizadas com sucesso.
201 Created	Recurso criado com sucesso.
204 No Content	Recurso excluído com sucesso.
400 Bad Request	Regra de negócio ou dado inválido.
404 Not Found	Recurso relacionado ou solicitado não encontrado.
422 Unprocessable Entity	Falha de validação do modelo de entrada.
401 Unauthorized	Previsto para autenticação futura.
403 Forbidden	Previsto para autorização futura.

Os erros de regra de negócio são retornados por HTTPException com mensagens no campo detail. A validação estrutural dos dados é realizada pelo Pydantic/FastAPI, resultando normalmente em 422 quando o corpo não atende ao modelo.

11. ARQUITETURA E ESTRUTURA DO PROJETO

A estrutura atual separa os modelos de entrada da API, a configuração de banco e os modelos ORM.

sistema_eventos_academicos/

├── main.py

├── model.py

├── database.py

├── db_models.py

├── requirements.txt

└── venv/

Arquivo	Responsabilidade
main.py	Rotas da API, regras de negócio e operações com banco.
model.py	Modelos Pydantic usados para entrada e validação dos dados.
database.py	Configuração do engine, sessão e Base do SQLAlchemy.
db_models.py	Modelos ORM que representam as tabelas PostgreSQL.
requirements.txt	Dependências principais do projeto.
venv/	Ambiente virtual Python utilizado para executar a aplicação.

Evolução arquitetural planejada: separar rotas, serviços, repositórios e testes em módulos próprios conforme o projeto crescer.

12. TECNOLOGIAS E JUSTIFICATIVAS

Tecnologia	Função	Justificativa
Python	Linguagem	Base do desenvolvimento da API.
FastAPI	Framework web	Criação de API REST, validação e documentação automática.
Pydantic	Validação/modelagem	Validação dos dados recebidos pela API.
Uvicorn	Servidor ASGI	Execução da aplicação FastAPI.
PostgreSQL	Banco relacional	Persistência dos dados e

		integridade referencial.
SQLAlchemy	ORM	Mapeamento entre classes Python e tabelas do banco.
psycopg	Driver PostgreSQL	Comunicação da aplicação Python com PostgreSQL.
Swagger/OpenAPI	Documentação/testes	Visualização e execução dos endpoints.

13. MATRIZ DE PERMISSÕES

A matriz abaixo representa a regra funcional planejada para os perfis. A autenticação/autorização poderá ser implementada em etapa posterior.

Operação	Administrador	Organizador	Participante
Consultar eventos	Sim	Sim	Sim
Cadastrar evento	Sim	Sim	Não
Atualizar evento	Sim	Sim	Não
Excluir evento	Sim	Sim*	Não
Gerenciar categorias	Sim	Não	Não
Gerenciar usuários	Sim	Não	Não
Realizar inscrição	Sim	Não	Sim
Cancelar própria inscrição	Sim	Não	Sim
Emitir certificado	Sim	Sim	Não
Consultar certificado próprio	Sim	Sim	Sim

* A autorização definitiva deve ser vinculada ao organizador responsável quando o mecanismo de autenticação for implementado.

14. BANCO DE DADOS E INTEGRIDADE

O projeto utiliza o banco PostgreSQL denominado eventos_academicos. A aplicação acessa o banco por meio do usuário de aplicação eventos_app, utilizando SQLAlchemy e o driver psycopg.

O PostgreSQL atua como servidor de banco de dados, enquanto o cliente psql pode ser utilizado para executar comandos SQL e administrar a estrutura. A aplicação Python utiliza o SQLAlchemy para abrir sessões e executar operações sobre as tabelas.

14.1 Restrições de integridade implementadas

- UNIQUE em usuarios.email.
- UNIQUE em organizadores.email.
- UNIQUE em categorias.nome.
- UNIQUE composto em inscricoes(usuario_id, evento_id).
- UNIQUE em certificados.codigo.
- Chaves estrangeiras entre as tabelas relacionadas.
- Validações de existência e vínculos também são aplicadas na API.

14.2 Tabelas

Tabela	Finalidade
eventos	Armazena eventos acadêmicos.
categorias	Armazena categorias.
usuarios	Armazena participantes e demais usuários.
inscricoes	Relaciona usuários e eventos.
certificados	Armazena certificados vinculados a inscrições.
organizadores	Armazena responsáveis pelos eventos.

15. FUNCIONALIDADES IMPLEMENTADAS

Módulo	Situação atual
Eventos	CRUD integrado ao PostgreSQL, validação de vagas, data, categoria, organizador e proteção contra exclusão com inscrições.
Categorias	CRUD, verificação de nome duplicado e proteção contra exclusão quando vinculada a evento.
Usuários	CRUD, verificação de e-mail duplicado e proteção contra exclusão quando possui inscrições.
Inscrições	CRUD, validação de usuário/evento, limite de vagas, status controlado, prevenção de duplicidade e proteção quando há certificado.
Certificados	CRUD, exigência de inscrição confirmada, código único e no máximo um certificado por inscrição.
Organizadores	CRUD, verificação de e-mail duplicado e proteção contra exclusão quando vinculado a eventos.

16. TESTES E VALIDAÇÃO

Os testes foram realizados pela interface Swagger/OpenAPI da FastAPI, com verificações de operações CRUD e regras de negócio.

16.1 Testes de regras realizados

Teste	Resultado esperado	Situação
-------	--------------------	----------

Cadastrar categoria com nome duplicado	400	Validado
Cadastrar usuário com e-mail duplicado	400	Validado
Cadastrar organizador com e-mail duplicado	400	Validado
Inscriver usuário duas vezes no mesmo evento	400	Validado
Inscriver além do limite de vagas	400	Validado
Criar certificado para inscrição Pendente	400 com mensagem de regra	Validado
Criar certificado para inscrição Confirmada	201	Validado
Criar certificado duplicado para mesma inscrição	400	Validado
Cadastrar certificado com código duplicado	400	Validado
Excluir evento com inscrições	400	Implementado
Excluir categoria vinculada a evento	400	Implementado
Reduzir vagas abaixo de inscrições ativas	400	Implementado

Um dos testes mais importantes foi a tentativa de emissão de certificado para inscrição com status Pendente. A API recusou a operação com a mensagem: "O certificado só pode ser emitido para uma inscrição confirmada." Após alterar a inscrição para Confirmada, a criação do certificado foi concluída com sucesso.

17. BACKLOG

- 23. Implementar autenticação de usuários.
- 24. Implementar autorização por perfil.
- 25. Adicionar JWT.
- 26. Separar rotas em módulos.
- 27. Criar camada de serviços.
- 28. Criar camada de repositórios.
- 29. Adicionar migrations com Alembic.
- 30. Melhorar tipos de data e horário no banco.
- 31. Adicionar filtros e paginação de eventos.
- 32. Adicionar testes automatizados com Pytest.
- 33. Adicionar endpoint específico para cancelamento de inscrição.
- 34. Adicionar consulta de inscrições por usuário e por evento.
- 35. Adicionar documentação de implantação.
- 36. Melhorar respostas padronizadas de erro.

18. CRITÉRIOS DE ACEITAÇÃO

- 37. O sistema permite cadastrar um evento válido e persistir os dados no PostgreSQL.
- 38. Não é possível cadastrar evento com quantidade de vagas menor ou igual a zero.
- 39. Não é possível cadastrar evento com categoria inexistente.
- 40. Não é possível cadastrar evento com organizador inexistente.
- 41. Não é possível duplicar e-mail de usuário.
- 42. Não é possível duplicar e-mail de organizador.
- 43. Não é possível duplicar nome de categoria.
- 44. Não é possível realizar duas inscrições do mesmo usuário no mesmo evento.
- 45. Não é possível ultrapassar a capacidade do evento com inscrições ativas.
- 46. Não é possível emitir certificado para inscrição não confirmada.

47. Uma inscrição não pode possuir mais de um certificado.
48. Os códigos de certificados não podem se repetir.
49. Os endpoints principais estão disponíveis no Swagger.
50. Os dados cadastrados pela API permanecem persistidos no PostgreSQL.
51. As relações entre as entidades são protegidas por chaves estrangeiras.

19. SITUAÇÃO ATUAL E PRÓXIMAS EVOLUÇÕES

O projeto possui uma API funcional integrada ao PostgreSQL, com seis entidades, operações CRUD, relacionamentos por chaves estrangeiras, validações de entrada, regras de negócio, restrições de integridade no banco e testes realizados pelo Swagger.

As principais evoluções restantes estão relacionadas à organização arquitetural, autenticação/autorização, migrations, testes automatizados, documentação de implantação e refinamentos que podem ser adicionados sem comprometer a estrutura atual.

Para a entrega acadêmica, recomenda-se anexar evidências do Swagger, do banco de dados e do repositório GitHub, conforme a orientação da atividade.

20. CONCLUSÃO

O Sistema de Gestão de Eventos Acadêmicos apresenta uma base funcional para gerenciamento de eventos e suas principais entidades relacionadas. A utilização do FastAPI permite disponibilizar endpoints REST e documentação interativa, enquanto PostgreSQL e SQLAlchemy fornecem persistência e integração com um banco relacional.

As regras de negócio implementadas reduzem inconsistências, como duplicidade de cadastros, inscrições repetidas, excesso de inscrições e emissão indevida de certificados. As chaves estrangeiras e restrições UNIQUE adicionadas ao PostgreSQL complementam essas proteções na camada de dados.

A estrutura desenvolvida atende à base funcional do projeto e permite sua evolução para autenticação, autorização, testes automatizados, migrations e uma arquitetura ainda mais modular.

21. REFERÊNCIAS

FASTAPI. **FastAPI Documentation**. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 2026.

PYDANTIC. **Pydantic Documentation**. Disponível em: <https://docs.pydantic.dev/>. Acesso em: 2026.

SQLALCHEMY. **SQLAlchemy Documentation**. Disponível em: <https://docs.sqlalchemy.org/>. Acesso em: 2026.

POSTGRESQL. **PostgreSQL Documentation**. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 2026.

PYTHON SOFTWARE FOUNDATION. **Python Documentation**. Disponível em: <https://docs.python.org/3/>. Acesso em: 2026.

